

3주차 예습과제- 딥러닝 파이토치 교과서 5장 - Part 1 (~5.2)

5장- 합성곱 신경망1

5.1 합성곱 신경망

💡 Intuition

사진을 한 번에 통째로 보는 대신, 작은 돋보기(필터)를 들고 사진 위를 한 칸씩 훑으면서 "여기 이런 무늬 있네"를 기록해 나가는 신경망이 CNN이다.

- 왜 필요한가 → 이미지를 펼치면(flatten) 공간 구조가 사라지고, 전체를 한 번에 계산하면 **자원·시간이 많이 든다**
- 어떻게 해결하나 → 이미지의 **국소적인 부분만** 계산하는 합성곱층 + 크기를 줄이는 풀링층
- 전체 구조 → 입력층 → 합성곱층 → 풀링층 → 완전연결층 → 출력층

역전파의 부담

- 순전파에서 계산된 오차 정보가 신경망의 **모든 노드**로 거꾸로 전달된다 (출력층 → 은닉층 → 입력층)
- 이 계산 과정은
 - 복잡하고
 - 자원(CPU 혹은 GPU, 메모리)을 많이 요구하며
 - 계산 시간도 오래 걸린다

합성곱 신경망의 해결 방식

- 이미지 **전체**를 한 번에 계산하지 않는다
- 이미지의 **국소적(local)** 부분을 계산한다
- 결과
 - 시간과 자원 절약
 - 이미지의 **세밀한 부분까지** 분석 가능

5.1.1 합성곱층의 필요성

- 합성곱 신경망은 **이미지·영상 처리**에 유용하다
- 예시 상황: 3×3 흑백(그레이스케일) 이미지가 있다고 가정 (교재 그림의 색은 설명용으로 입힌 것)

기존 방식(완전연결)의 이미지 분석

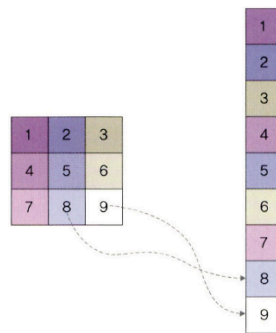
1. 3×3 배열을 한 줄로 **펼친다(flattening)** → 길이 9짜리 벡터

2. 각 픽셀에 가중치를 곱한다
3. 은닉층으로 전달한다

문제점

- 펼치는 순간 데이터의 공간적 구조가 무시된다
 - 원래 이미지에서 8의 바로 옆은 9, 바로 위는 5였는데, 한 줄로 펴면 "위·아래 이웃" 관계가 사라짐
- 이 공간 구조 손실을 막으려고 도입된 것이 합성곱층

♥ 그림 5-1 합성곱층 원리



5.1.2 합성곱 신경망 구조

정의

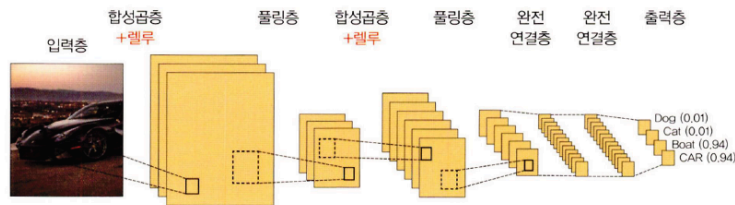
- 합성곱 신경망 (Convolutional Neural Network, **CNN** 또는 **ConvNet**)
- 주 용처: 음성 인식, 이미지/영상 인식
- 다차원 배열 데이터를 처리하도록 구성됨 → 컬러 이미지 같은 다차원 배열 처리에 특화

계층 다섯 개

순서	계층	역할 요약
1	입력층	이미지 데이터가 최초로 거치는 계층 (높이·너비·채널의 3차원 데이터)
2	합성곱층	커널/필터로 입력 데이터의 특성 추출 → 특성 맵 생성
3	풀링층	특성 맵을 다운 샘플링해 연산량 감소, 주요 특성 벡터 추출
4	완전연결층	3차원 특성 맵을 1차원 벡터로 펼침(flatten)
5	출력층	소프트맥스로 각 레이블에 속할 확률 출력

전체 동작 흐름

▼ 그림 5-2 합성곱 신경망 구조



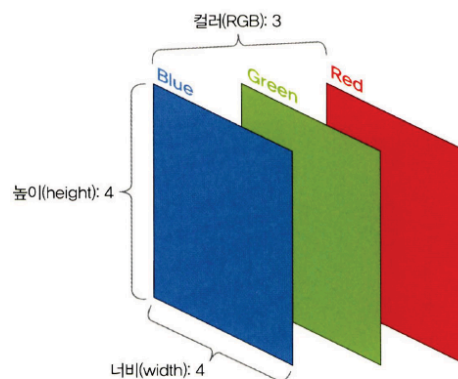
- 합성곱층 + 풀링층을 거치며 입력 이미지의 주요 특성 벡터(feature vector) 추출
- 추출된 특성 벡터들이 완전연결층을 거치며 1차원 벡터로 변환
- 출력층에서 활성화 함수 소프트맥스(softmax) 로 최종 결과 출력

입력층 (input layer)

- 입력 이미지 데이터가 최초로 거치는 계층
- 이미지는 단순 1차원 데이터가 아니라 3차원 데이터
 - 높이 (height)
 - 너비 (width)
 - 채널 (channel)

이미지 종류	채널 값
그레이스케일 (gray scale)	1
컬러 (RGB)	3

▼ 그림 5-3 채널



💡 컬러 사진 = 빨강 필름 + 초록 필름 + 파랑 필름 세 장을 겹쳐 놓은 것. 필름 한 장이 채널 하나.

합성곱층 (convolutional layer)

역할과 용어

- 역할: 입력 데이터에서 특성을 추출

용어	의미
커널(kernel) / 필터(filter)	이미지의 특성을 감지하기 위해 쓰는 작은 가중치 행렬. 이미지의 모든 영역을 훑으면서 특성을 추출
특성 맵 (feature map)	커널/필터가 훑고 지나가며 추출한 결과물
스트라이드 (stride)	필터를 적용하는 위치의 간격 . 커널은 지정된 스트라이드 간격만큼 순차적으로 이동

- 커널 크기는 **3×3, 5×5**가 일반적

연산 방법

입력 이미지와 필터를 **포개 놓고**, 같은 자리에 있는 숫자끼리 **곱한 뒤 모두 더한다** (가중합)

계산 예제 — 스트라이드 1, 입력 (6, 6, 1), 필터 3×3

입력 (6×6)

```
1 0 0 0 0 1
0 1 0 0 1 0
0 0 1 1 0 0
1 0 0 0 1 0
0 1 0 0 1 0
0 0 1 0 1 0
```

필터/커널 (3×3)

```
1 0 1
0 1 0
1 0 1
```

1단계. 입력 이미지에 3×3 필터 적용 (1~3행, 1~3열)

```

검친 영역      필터
1 0 0          1 0 1
0 1 0    ×    0 1 0
0 0 1          1 0 1

```

$$(1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) = 3$$

2단계. 필터가 1만큼 이동 (1~3행, 2~4열)

```

0 0 0
1 0 0
0 1 1

```

$$(0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) = 1$$

3단계. 필터가 1만큼 두 번째 이동 (1~3행, 3~5열)

```
0 0 0
0 0 1
1 1 0
```

$$(0 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) + (1 \times 0) + (0 \times 1) = 1$$

4단계. 필터가 1만큼 세 번째 이동 (1~3행, 4~6열)

```
0 0 1
0 1 0
1 0 0
```

$$(0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) = 3$$

→ 첫 번째 행 완성: **3 1 1 3**

5단계. 필터가 1만큼 네 번째 이동 (오른쪽 끝에 닿았으므로 한 줄 아래로: 2~4행, 1~3열)

```
0 1 0
0 0 1
1 0 0
```

$$(0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) = 1$$

6단계. 필터가 1만큼 마지막으로 이동 (4~6행, 4~6열)

```
0 1 0
0 1 0
0 1 0
```

$$(0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (0 \times 1) = 1$$

최종 출력 (특성 맵 4x4)

```
3 1 1 3
1 2 3 0
1 2 2 2
3 0 3 1
```

예제 정리

- 이미지 크기 **(6, 6, 1)**, 커널/필터 **3x3**, 스트라이드 **1**
- 커널은 스트라이드 간격만큼 순회하면서 **모든 입력 값과 합성곱 연산** → 새로운 특성 맵 생성
- 커널과 스트라이드의 상호 작용으로 원본 **(6, 6, 1)** → **(4, 4, 1)** 크기의 특성 맵으로 **줄어듦**
- 공식 계산: $(6 - 3 + 2 \times 0) / 1 + 1 = 4$ ✓

계산 예제 — 입력 3×3×3, 필터 2×2×3

채널 1

입력		필터		결과
1 1 0		2 0		-2 0
2 2 0	×	-1 -1	=	3 2
0 1 1				

채널 2

입력		필터		결과
2 0 1		1 1		2 0
1 0 1	×	0 -1	=	0 0
2 1 1				

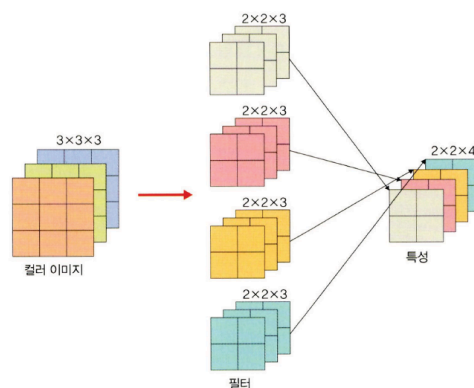
채널 3

입력		필터		결과
1 1 1		-1 0		-2 1
1 2 0	×	1 -1	=	-2 -3
0 1 2				

세 결과를 같은 자리끼리 더함 → 특성 맵 (2×2)

$$\begin{array}{rcl} (-2)+2+(-2) & 0+0+1 & -2 \quad 1 \\ 3+0+(-2) & 2+0+(-3) & = \quad 1 \quad -1 \end{array}$$

▼ 그림 5-11 필터가 2 이상인 합성곱



6-3. 필터가 2개 이상인 합성곱

- 필터가 두 개 이상이면 필터 각각이 특성 추출 결과의 채널이 된다
- 각 계산은 앞에서 진행한 방법과 동일
- 예시: 컬러 이미지 3×3×3 에 2×2×3 필터 4개 적용 → 특성 2×2×4
 - 가로·세로: $(3 - 2)/1 + 1 = 2$

- 깊이: 필터 개수 = 4

💡 필터 하나 = "특정 무늬 하나만 찾는 탐지기". 탐지기를 4개 돌리면 탐지 결과지도 4장 → 출력 채널 4.

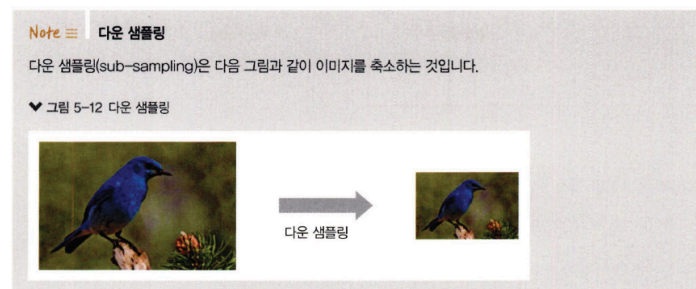
합성곱층 요약

구분	항목	기호 / 식
입력 데이터	크기	$W_1 \times H_1 \times D_1$ (W_1 : 가로, H_1 : 세로, D_1 : 채널 또는 깊이)
하이퍼파라미터	필터 개수	K
하이퍼파라미터	필터 크기	F
하이퍼파라미터	스트라이드	S
하이퍼파라미터	패딩	P
출력 데이터	가로	$W_2 = (W_1 - F + 2P) / S + 1$
출력 데이터	세로	$H_2 = (H_1 - F + 2P) / S + 1$
출력 데이터	깊이	$D_2 = K$

풀링층 (pooling layer)

역할

- 합성곱층과 유사하게 특성 맵의 차원을 **다운 샘플링**하여 **연산량을 감소**시킴
- **주요한 특성 벡터를 추출**하여 학습을 효과적으로 할 수 있게 함



종류	방식
최대 풀링 (max pooling)	대상 영역에서 최댓값 을 추출
평균 풀링 (average pooling)	대상 영역에서 평균 을 반환

- 대부분의 합성곱 신경망에서는 **최대 풀링**을 사용
- 이유: 평균 풀링은 각 커널 값을 **평균화**시켜 **중요한 가중치**를 갖는 값의 특성이 **희미해질** 수 있기 때문

최대 풀링 계산 예제 — 입력 4×4, 영역 2×2, 스트라이드 2

입력

```

3  -1 12  -1
-3  1   0   1
 2  -3   0   1
 3  -2   4  -1

```

단계	대상 영역(위치)	값	선택된 최댓값
첫 번째	왼쪽 위	3, -1, -3, 1	3
두 번째	오른쪽 위	12, -1, 0, 1	12
세 번째	왼쪽 아래	2, -3, 3, -2	3
네 번째	오른쪽 아래	0, 1, 4, -1	4

최대 풀링 출력

```
3 12
3  4
```

평균 풀링 계산

- 최대 풀링과 유사한 방식으로 진행하되, 각 영역의 **평균**으로 계산

```
0 = (3 + (-1) + (-3) + 1) / 4
3 = (12 + (-1) + 0 + 1) / 4
0 = (2 + (-3) + 3 + (-2)) / 4
1 = (0 + 1 + 4 + (-1)) / 4
```

평균 풀링 출력

```
0 3
0 1
```

최대 풀링 vs 평균 풀링 결과 비교

위치	최대 풀링	평균 풀링
왼쪽 위	3	0
오른쪽 위	12	3
왼쪽 아래	3	0
오른쪽 아래	4	1

→ 12처럼 튀는 값이 평균 풀링에서는 3으로 희석되는 것을 확인할 수 있음

풀링층 요약

(최대 풀링과 평균 풀링은 계산 과정만 다르고 **사용하는 파라미터는 동일**)

구분	항목	기호 / 식
입력 데이터	크기	$W_1 \times H_1 \times D_1$
하이퍼파라미터	필터 크기	F
하이퍼파라미터	스트라이드	S
출력 데이터	가로	$W_2 = (W_1 - F) / S + 1$
출력 데이터	세로	$H_2 = (H_1 - F) / S + 1$
출력 데이터	깊이	$D_2 = D_1$

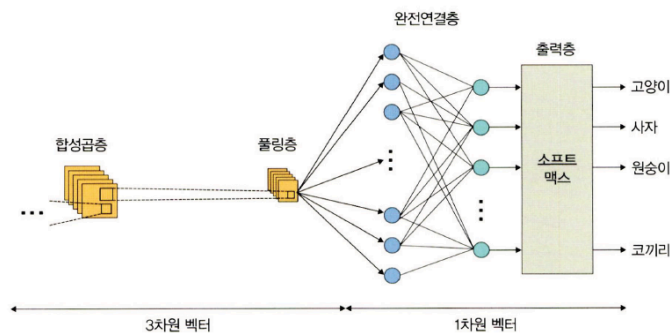
합성곱층 vs 풀링층 비교

항목	합성곱층	풀링층
하는 일	특성 추출 (가중합)	다운 샘플링 (최댓값/평균)
하이퍼파라미터	K, F, S, P	F, S
패딩 P	있음	없음
출력 가로·세로	$(W_1 - F + 2P)/S + 1$	$(W_1 - F)/S + 1$
출력 깊이	$D_2 = K$ (필터 개수)	$D_2 = D_1$ (그대로 유지)
학습되는 가중치	있음 (필터 값)	없음

완전연결층 (fully connected layer)

- 합성곱층과 풀링층을 거치면서 **차원이 축소된 특성 맵**이 최종적으로 전달되는 곳
- 이 과정에서 이미지는 **3차원 벡터 → 1차원 벡터로 펼쳐진다(flatten)**

▼ 그림 5-18 완전연결층



출력층 (output layer)

- 소프트맥스 활성화 함수** 사용 → 입력받은 값을 **0~1 사이의 값**으로 출력
- 이미지가 **각 레이블(label)에 속할 확률 값**이 출력됨
- 이때 **가장 높은 확률 값을 갖는 레이블**이 최종 값으로 선정

5.1.3 1D, 2D, 3D 합성곱

분류 기준

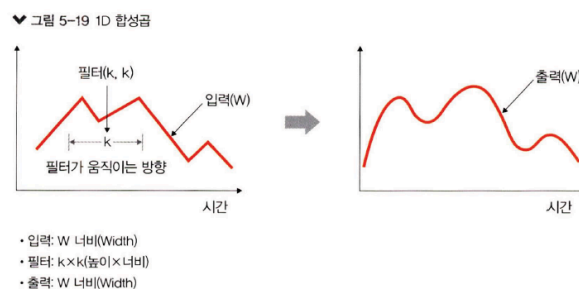
- 합성곱은 다음 두 가지에 따라 1D, 2D, 3D로 분류한다
 - 필터가 **이동하는 방향의 수**
 - 출력 형태**

종류	입력	필터	필터 이동 방향	출력 형태	대표 사례 / 용도
1D 합성곱	W	k	1개 (시간 축 좌우)	1D 배열 (W)	그래프 곡선 완화
2D 합성곱	(W, H)	(k, k)	2개	2D 행렬 (W, H)	—
3D 합성곱	(W, H, L)	(k, k, d), $d < L$	3개	3D (W, H, L)	—

종류	입력	필터	필터 이동 방향	출력 형태	대표 사례 / 용도
3D 입력을 갖는 2D 합성곱	(W, H, L)	(k, k, L)	2개	2D 행렬 (W, H)	LeNet-5, VGG
1×1 합성곱	(W, H, L)	(1, 1, L)	2개	2D 행렬 (W, H)	GoogLeNet (채널 수 조정 → 연산량 감소)

1D 합성곱

- 필터가 **시간을 축으로 좌우로만** 이동할 수 있는 합성곱
- 입력(W)과 필터(k)에 대한 출력은 **W**가 된다 → 출력 형태는 **1D 배열**
- 용도: **그래프 곡선을 완화**할 때 많이 사용



예시

- 입력: [1, 1, 1, 1, 1]
- 필터: [0.25, 0.5, 0.25]
- 출력: [1, 1, 1]

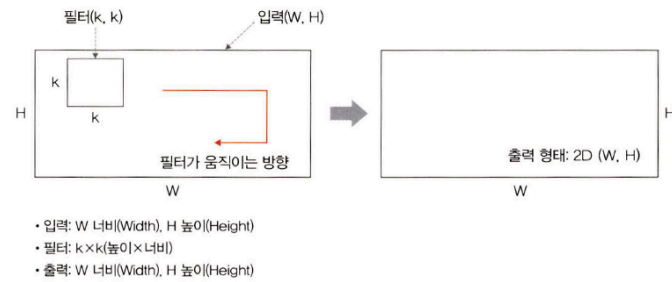
$$\begin{aligned}
 [1, 1, 1], 1, 1 &\rightarrow 1 \times 0.25 + 1 \times 0.5 + 1 \times 0.25 = 1 \\
 1, [1, 1, 1], 1 &\rightarrow 1 \times 0.25 + 1 \times 0.5 + 1 \times 0.25 = 1 \\
 1, 1, [1, 1, 1] &\rightarrow 1 \times 0.25 + 1 \times 0.5 + 1 \times 0.25 = 1
 \end{aligned}$$

💡 필터 [0.25, 0.5, 0.25]는 "나 50% + 양옆 25%씩" 섞는 가중 평균 → 뾰족뾰족한 꺾은선 그래프가 부드러운 곡선으로 바뀜 (그림 5-19의 왼쪽 → 오른쪽)

2D 합성곱

- 필터가 **방향 두 개로** 움직이는 형태
- 입력 (W, H) 과 필터 (k, k) 에 대한 출력은 **(W, H)**
- 출력 형태는 **2D 행렬**

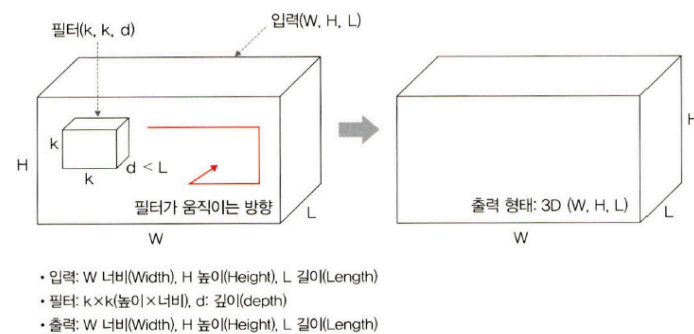
▼ 그림 5-20 2D 합성곱



3D 합성곱

- 필터가 움직이는 방향이 **세 개**
- 입력 (W, H, L) 에 필터 (k, k, d) 를 적용 $\rightarrow$ 출력 (**W, H, L**)
- 출력은 **3D 형태**
- ⚠ 이때 **$d < L$** 을 유지하는 것이 중요

▼ 그림 5-21 3D 합성곱

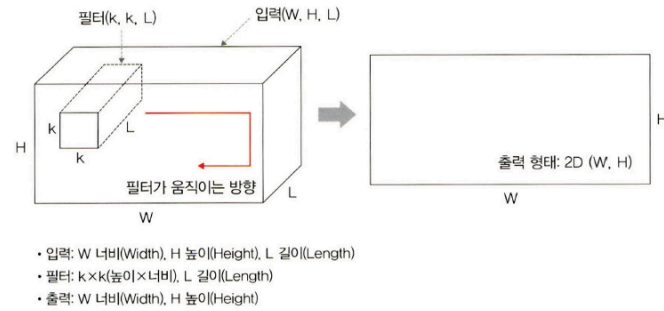


💡 왜 **$d < L$** 이어야 하나: 필터 깊이가 입력 길이보다 작아야 **깊이 방향으로도 움직일 여유**가 생긴다. $d = L$ 이면 깊이 방향으로 꽉 끼어서 못 움직임

3D 입력을 갖는 2D 합성곱

- 입력이 ($224 \times 224 \times 3$, $112 \times 112 \times 32$) 와 같은 **3D 형태임에도**, 출력 형태가 3D가 아닌 **2D 행렬**을 취하는 것
- 이유: 필터의 길이(L)가 입력 채널의 길이(L)와 같아야 하기 때문
- 입력 (W, H, L) 에 필터 (k, k, L) 를 적용 $\rightarrow$ 출력 (**W, H**)
- 필터는 **두 방향으로** 움직이며 출력 형태는 **2D 행렬**
- 대표적 사례: **LeNet-5, VGG**

▼ 그림 5-22 3D 입력을 갖는 2D 합성곱



💡 **앞 내용과 연결:** 그림 5-10의 **컬러 이미지 합성곱**(입력 $3 \times 3 \times 3$, 필터 $2 \times 2 \times 3 \rightarrow$ 특성 맵 2×2)이 정확히 이 경우. 입력은 3D인데 필터 채널 수가 입력 채널 수와 같아서, 채널별 결과를 더하면 **2D 한 장**이 나온다. 우리가 보통 "CNN"이라고 부르는 건 대부분 이 방식!!

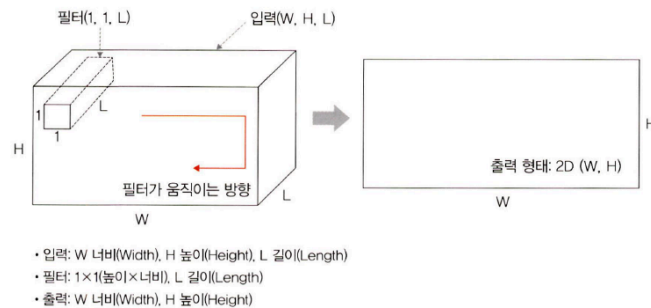
3D 합성곱 vs 3D 입력을 갖는 2D 합성곱

항목	3D 합성곱	3D 입력을 갖는 2D 합성곱
입력	(W, H, L)	(W, H, L)
필터	(k, k, d)	(k, k, L)
필터 깊이 조건	$d < L$	필터 길이 = 입력 채널 길이 (L)
이동 방향	3개	2개
출력	3D (W, H, L)	2D (W, H)

1×1 합성곱

- **3D 형태로 입력된다**
- 입력 (W, H, L) 에 필터 (1, 1, L) 를 적용 $\rightarrow$ 출력 (W, H)
- 효과: **채널 수를 조정해서 연산량이 감소**
- 대표적 사례: **GoogLeNet**

▼ 그림 5-23 1×1 합성곱



5.2 합성곱 신경망 맛보기

1. fashion_mnist 데이터셋

항목	내용
출처	토치비전(torchvision)에 내장된 예제 데이터
내용	운동화, 셔츠, 샌들 같은 작은 이미지의 모음
클래스 수	열 가지 (기본 MNIST처럼)
이미지 크기	28×28 픽셀
이미지 수	7만 개 (훈련 6만 + 테스트 1만)
훈련 데이터 (train_images)	0~255 사이 값을 갖는 28×28 넘파이(NumPy) 배열
레이블 데이터 (train_labels)	0~9 정수 값을 갖는 배열 (이미지의 클래스)

레이블 ↔ 클래스

레이블	클래스	레이블	클래스
0	T-Shirt	5	Sandal
1	Trouser	6	Shirt
2	Pullover	7	Sneaker
3	Dress	8	Bag
4	Coat	9	Ankle Boot

2. 코드 5-1 ~ 5-2. 라이브러리 호출과 장치 확인

2-1. 라이브러리

라이브러리	용도
numpy, matplotlib.pyplot	배열 계산, 이미지 출력
torch, torch.nn (nn)	파이토치 본체, 딥러닝 모델 구성 모듈 모음
torch.autograd.Variable	자동 미분(Autograd)용 래퍼
torch.nn.functional (F)	활성화 함수·합성곱 등 함수형 API
torchvision, torchvision.transforms	내장 데이터셋, 데이터 전처리
torch.utils.data.Dataset, DataLoader	데이터를 배치 단위로 불러오기

2-2. CPU 혹은 GPU 장치 확인

- 파이토치는 기본적으로 **GPU 사용을 권장**하지만, GPU가 없는 환경에서도 정상 실행됨
- GPU가 장착되어 있고 설정이 되어 있으면 파이토치가 **자동으로 인식**. 교재는 GPU 설정을 안 했으므로 CPU 사용

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

→ "GPU 있으면 cuda:0, 없으면 cpu"

2-3. Note — GPU 사용

상황	코드	비고
GPU 1개	<code>device = torch.device("cuda:0" ...) → model = Net() → model.to(device)</code>	모델을 장치로 보냄
GPU 여러 개	<code>device = torch.device("cuda" ...) → if torch.cuda.device_count() > 1: model = nn.DataParallel(model) → model.to(device)</code>	<code>nn.DataParallel</code> 사용

- `nn.DataParallel` 은 배치 크기가 알아서 각 GPU로 분배되는 방식 → GPU 수만큼 배치 크기도 늘려 주어야 함

3. 코드 5-3 ~ 5-4. 데이터 내려받기와 데이터로더

3-1. torchvision.datasets.FashionMNIST

- `torchvision.datasets` 는 `torch.utils.data.Dataset` 의 하위 클래스로 다양한 데이터셋(CIFAR, COCO, MNIST, ImageNet 등)을 포함

```
torchvision.datasets.FashionMNIST("../chap05/data", download=True,
                                   transform=transforms.Compose([transforms.ToTensor()])))
```

파라미터	의미
㉠ 첫 번째 파라미터	FashionMNIST를 내려받을 위치
㉡ download=True	첫 번째 파라미터 위치에 데이터셋이 있는지 확인한 후 (없으면) 내려받음
㉢ transform	이미지를 텐서(0~1)로 변경
train=False	테스트 데이터셋을 내려받음 (기본값 train=True → 훈련 데이터)

- 내려받은 결과는 `../chap05/data/FashionMNIST` 폴더 하위에서 확인 (환경에 따라 위치가 다를 수 있음)
- 다운로드 로그: train-images / train-labels / t10k-images / t10k-labels 4개 파일을 Downloading → Extracting → Processing... → Done!

3-2. torch.utils.data.DataLoader

- 원하는 크기의 배치 단위로 데이터를 불러오거나, 순서가 무작위로 섞이도록(shuffle) 할 수 있음

```
torch.utils.data.DataLoader(train_dataset, batch_size=100)
```

파라미터	의미
㉠ 첫 번째 파라미터	데이터를 불러올 데이터셋
㉡ batch_size=100	데이터를 100개 단위로 묶어서 불러옴

💡 6만 장을 한 번에 넣으면 메모리가 터진다. 100장씩 묶어서 넣으면 1에포크 = 60000 / 100 = **600번** 반복

4. 코드 5-5. 이미지 20개 확인 + 파이썬 기초 보충

4-1. 코드 요약

- `labels_map` : 0~9 정수를 클래스 이름으로 바꾸는 딕셔너리 (열 개의 클래스)

- `plt.figure(figsize=(8,8))` : 출력할 이미지의 가로세로 길이, 단위는 inch
- `columns=4, rows=5` → $4 \times 5 = 20$ 개 이미지를 격자로 출력

4-2. ① np.random 계열

함수	의미
<code>np.random</code>	무작위로 데이터를 생성할 때 사용
<code>np.random.randint()</code>	이산형 분포 를 갖는 데이터에서 무작위 표본 추출
<code>np.random.randint(len(train_dataset))</code>	0 ~ (train_dataset의 길이) 값을 갖는 분포에서 랜덤 숫자 한 개
<code>np.random.randint(10)</code>	0~9 임의의 정수
<code>np.random.randint(1, 10)</code>	1~9 임의의 정수
<code>np.random.rand(8)</code>	0~1 사이 난수를 (1×8)로 출력 (교재 주석은 "정규표준분포"라 하지만 실제로는 균일분포)
<code>np.random.rand(4, 2)</code>	0~1 사이 난수를 (4×2)로 출력
<code>np.random.randn(8)</code>	평균 0, 표준편차 1인 가우시안 정규분포 난수를 (1×8)로 출력
<code>np.random.randn(4, 2)</code>	가우시안 정규분포 난수를 (4×2)로 출력

4-3. ② 배열 인덱싱 — `train_dataset[img_xy][0][0,:,:]` 이해하기

2차원 예시 (`examp = np.arange(0, 100, 3); examp.resize(6, 4)` → 0~99에서 3씩 건너뛴 값을 6×4로)

```
[[ 0  3  6  9]
 [12 15 18 21]
 [24 27 30 33]
 [36 39 42 45]
 [48 51 54 57]
 [60 63 66 69]]
```

표현	결과	의미
<code>examp[3]</code>	[36 39 42 45]	3행에 해당하는 모든 요소 (행·열은 0부터 시작)
<code>examp[3, 3]</code>	45	3행의 3번째 열 값
<code>examp[3][3]</code>	45	위와 동일

3차원 예시 (`examp = np.arange(0, 500, 3); examp.resize(3, 5, 5)` → 3덩어리 × 5행 × 5열)

- `examp[2][0][3]` → 2번 덩어리의 0행 3열 = **159**

그래서 `train_dataset[img_xy][0][0,:,:]` 는

단계	표현	결과
1	<code>train_dataset[img_xy]</code>	(이미지 텐서, 레이블) 튜플
2	<code>[0]</code>	이미지 텐서, 크기 [1, 28, 28] (채널, 높이, 너비)
3	<code>[0, :, :]</code>	0번 채널의 모든 행·모든 열 → [28, 28] 2차원 (imshow에 넣기 위해)
—	<code>train_dataset[img_xy][1]</code>	레이블 (0~9 정수) → labels_map으로 이름 변환

5. 코드 5-6. 심층 신경망(DNN) 모델 생성

CNN과 비교하기 위해 **ConvNet이 적용되지 않은 네트워크**(완전연결층만)를 먼저 만든다

5-1. 구조

```
입력 [100, 1, 28, 28]
→ view(-1, 784)          [100, 784]    ← 28×28 = 784로 펼침
→ fc1 (784 → 256) + ReLU
→ Dropout(0.25)
→ fc2 (256 → 128) + ReLU
→ fc3 (128 → 10)         ← 클래스 10개
```

5-2. 번호별 설명

번호	항목	설명
①	class FashionDNN(nn.Module) / init	클래스 형태의 모델은 항상 torch.nn.Module을 상속 . <code>__init__()</code> 은 객체가 갖는 속성 값을 초기화하며 객체가 생성 될 때 자동 호출 . <code>super(FashionDNN, self).__init__()</code> 은 부모(super) 클래스를 상속받겠다는 뜻
②	nn.Linear(in_features=784, out_features=256)	nn은 딥러닝 모델 구성에 필요한 모듈 패키지, Linear는 단순 선형 회귀 모델을 만들 때 사용. ① in_features: 입력의 크기 / ② out_features: 출력의 크기. forward()에는 첫 번째 파라미터 값만 넘겨주고, 두 번째 파라미터 크기가 forward() 연산 결과가 됨
③	nn.Dropout(0.25)	p만큼의 비율로 텐서 값이 0이 되고, 0이 되지 않는 값은 기존 값에 1/(1-p)만큼 곱해져 커짐 . 예: p=0.3 → 전체 값 중 0.3의 확률로 0, 나머지 0.7은 1/(1-0.7)배
④	forward()	모델이 학습 데이터를 입력받아 순전파(forward propagation) 를 진행. 반드시 forward 라는 이름이어야 하며, 객체를 데이터와 함께 호출하면(<code>model(train)</code>) 자동 실행. 순전파 = $H(x)$ 식에 입력 x를 넣어 예측 y를 얻는 것 ($H(x) = \text{sigmoid}(x_1w_1 + x_2w_2 + b)$ 같은 형태, 모델마다 다름)
⑤	input_data.view(-1, 784)	파이토치의 view = 넘파이의 reshape . (?, 784) 크기로 변경. -1은 "사용자가 잘 모르겠으니 파이토치에 맡기겠다", 두 번째 차원은 784로. 즉 2차원 텐서 (?, 784)로 변경
⑥	F.relu()	활성화 함수 지정 방법 두 가지: F.relu() 는 forward()에서 정의 / nn.ReLU() 는 init()에서 정의

💡 **Dropout** : 시험 공부할 때 매번 문제집 25%를 가리고 푸는 것. 특정 문제(뉴런)에만 의존하지 않게 되어 **과적합을 막는다**. 살아남은 값을 1/(1-p)배 키우는 건 전체 합의를 크기를 유지하기 위해서

5-3. 📝 Note — 객체 (Object)

- 파이토치의 근간은 **C++** → C++의 객체 지향 프로그래밍 특징을 파이토치에서도 사용
- 객체 지향 프로그래밍(OOP)**: 필요한 데이터를 추상화하여 속성이나 행동·동작·특징 등을 객체로 만들고, 그 객체들이 서로 유기적으로 동작하도록 하는 방법
- 쉽게 말해 **클래스 = 붕어빵 틀, 객체 = 붕어빵** → **재사용성** 때문에 많이 사용
- 객체(object)란 메모리를 할당받아 프로그램에서 사용되는 모든 데이터 → 변수, 함수 등도 모두 객체
- 사용법: `객체명 = 클래스명()`

5-4. 📝 Note — 클래스와 함수

구분	함수 (function)	클래스 (class)
정의	하나의 특정 작업을 수행하기 위해 독립적으로 설계된 코드	함수뿐 아니라 관련 변수까지 한꺼번에 묶어서 관리·재사용하게 해 주는 것
결과값	호출은 작업만 수행 → 결과를 계속 쓰려면 따로 저장 해야 함	<code>self.result</code> 처럼 객체 안에 저장됨
재사용	함수 + 데이터 저장 변수까지 같이 관리해야 함	객체 단위로 통째로 재사용

코드 비교

함수 방식	클래스 방식
<code>def add(num1, num2): return num1 + num2</code>	<code>class Calc: __init__에서 self.result = 0, add(self, num1, num2)에서 self.result 갱신</code>
<code>print(add(1, 2)) → 3 / print(add(2, 3)) → 5</code>	<code>obj1 = Calc(), obj2 = Calc() 두 객체 생성</code>
	<code>obj1.add(1, 2) → 3, obj1.add(2, 3) → 5 / obj2.add(2, 2) → 4, obj2.add(2, 3) → 5</code>

- `__init__(self)` : 객체를 생성할 때 호출하면 실행되는 **초기화 함수**
- 결과에서 보듯 **두 객체는 독립적으로 연산**됨. 개별 함수로 구현했다면 복잡했을 코드가 클래스로 간결해짐

5-5. nn.xx 와 nn.functional.xx (F.xx)

- 활성화 함수만 놓고 보면 차이는 "사용하는 위치"(forward vs **init**)뿐이지만, 근본적으로는 **사용 방법**이 다름

표 5-1 비교

구분	nn.xx	nn.functional.xx
형태	nn.Conv2d: 클래스 . nn.Module 클래스를 상속받아 사용	nn.functional.conv2d: 함수 . def function(input)으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후, 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터·데이터를 함께 전달
위치	nn.Sequential 내에 위치 가능	nn.Sequential에 위치할 수 없음
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

코드 비교

nn 사용	nn.functional 사용
<code>inputs = torch.randn(64, 3, 244, 244)</code>	<code>inputs = torch.randn(64, 3, 244, 244)</code>
<code>conv = nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, padding=1)</code>	<code>weight = torch.randn(64, 3, 3, 3) / bias = torch.randn(64)</code>
<code>outputs = conv(inputs)</code>	<code>outputs = F.conv2d(inputs, weight, bias, padding=1)</code>

- nn.Conv2d는 input_channel-output_channel로 연산하지만, functional.conv2d는 **입력(input)과 가중치(weight) 자체를 직접** 넣어 줌 → 가중치를 전달할 때마다 **새로 정의**해야 함. 그 외 채워야 하는 파라미터는 비슷
- 예시 주석: 세 개의 채널이 입력되어 64개의 채널이 출력되기 위한 연산으로 3x3 커널 사용 → 출력 [64, 64, 244, 244]

💡 **직관:** nn.Conv2d는 "가중치를 스스로 갖고 있는 부품", F.conv2d는 "가중치를 매번 손에 들고 와야 하는 도구". 모델 안에 넣어 학습시킬 땐 부품(nn.xx), 일회성 계산은 도구(F.xx)

6. 코드 5-7. 손실 함수·학습률·옵티마이저 정의

항목	코드	설명
학습률	learning_rate = 0.001	
모델	model = FashionDNN(); model.to(device)	객체 생성 후 장치로 이동
손실 함수	criterion = nn.CrossEntropyLoss()	분류 문제에서 사용하는 손실 함수
옵티마이저	optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)	① 경사 하강법은 Adam , lr = 0.001

- `print(model)` 출력:

```
FashionDNN(
  (fc1): Linear(in_features=784, out_features=256, bias=True)
  (drop): Dropout(p=0.25, inplace=False)
  (fc2): Linear(in_features=256, out_features=128, bias=True)
  (fc3): Linear(in_features=128, out_features=10, bias=True)
)
```

7. 코드 5-8. 심층 신경망을 이용한 모델 학습

7-1. 학습 루프 구조

```
for epoch (5회):
    for images, labels in train_loader (600회):          ← 배치 100개씩
        images, labels → device
        train = Variable(images.view(100, 1, 28, 28))
        outputs = model(train)                          ← 순전파
        loss = criterion(outputs, labels)
        optimizer.zero_grad()                           ← 기울기 초기화
        loss.backward()                                  ← 역전파
        optimizer.step()                                 ← 가중치 업데이트
        count += 1
        if count가 50의 배수: 테스트 데이터 전체로 정확도 계산 → 리스트에 저장
        if count가 500의 배수: Iteration / Loss / Accuracy 출력
```

7-2. 번호별 설명

번호	항목	설명
①, ①'	loss_list = []append()	리스트 사용법: ① 비어 있는 배열을 만들고 → ①' append 메서드로 데이터를 하나씩 추가
②	for images, labels in train_loader:	for 구문으로 레코드(행)를 하나씩 가져옴. <code>for x, y in train:</code> 처럼 in 앞에 변수 두 개를 두면 레코드에서 요소 두 개를 꺼내 오겠다 는 의미 (그림 5-26: [[10,20],[30,40],[50,60]] → x=10, y=20)
③	images.to(device), labels.to(device)	모델이 데이터를 처리하려면 모델과 데이터가 동일한 장치(CPU 또는 GPU)에 있어야 함 . 코드 5-7에서 model.to(device)가 GPU였다면 데이터도 GPU로. CPU 데이터를 GPU 모델에 넣거나 반대면 런타임 오류
④	Variable(images.view(100, 1, 28, 28))	Autograd = 자동 미분을 수행하는 파이토치 핵심 패키지. 미분 값을 저장하기 위해 테이프(tape) 사용 → 순전파(forward) 단계에서 모든 연산을 저장, 역전파(backward) 단계에서 꺼내 사용. Variable을 사용해 역전파용 미분 값을 자동 계산
⑤	accuracy = correct * 100 / total	분류 정확도 = 정확한 예측 / 전체 예측 (×100 → 백분율)

- `torch.max(outputs, 1)[1]` : 10개 클래스 점수 중 최댓값의 **인덱스**(= 예측 클래스). `[0]` 은 최댓값, `[1]` 은 위치
- `if not (count % 50)` : `count % 50` 이 0이면 `not 0` = True → **50의 배수마다 실행**

7-3. 정확도와 오류율

지표	식
분류 정확도	classification accuracy = correct predictions / total predictions
백분율	classification accuracy = correct predictions / total predictions × 100
오분류율(오류율)	error rate = (1 - (correct predictions / total predictions)) × 100

7-4. ⚠ 클래스가 세 개 이상일 때 정확도 해석 주의

- 정확도 80% → 그 값이 **모든 클래스가 동등하게** 고려된 것인지, **특정 클래스만** 잘 맞춘 것인지 알 수 없음
- 정확도 90% → 100개 중 90개가 **하나의 클래스**에 속한다면, 모든 데이터를 그 클래스로 찍어도 90% → 높다고 할 수 없음
- ⇒ **데이터 특성(클래스 분포)에 따라 정확도를 잘 관측**해야 함

💡 **직관**: 환자 100명 중 90명이 정상일 때 "전원 정상"이라고 답해도 정확도 90%. 의료 AI에서 정확도만 보면 안 되는 이유가 여기서 나옴

7-5. 실행 결과 (교재)

Iteration	Loss	Accuracy
500	0.5628	82.71%
1000	0.4662	84.40%
1500	0.3302	84.41%

Iteration	Loss	Accuracy
2000	0.3132	85.06%
2500	0.2840	86.26%
3000	0.3125	86.48%

→ 최종 정확도 약 **86%**. 5에포크 × 600반복 = 3000 iteration

8. 코드 5-9. 합성곱 네트워크(CNN) 생성

8-1. 구조

```

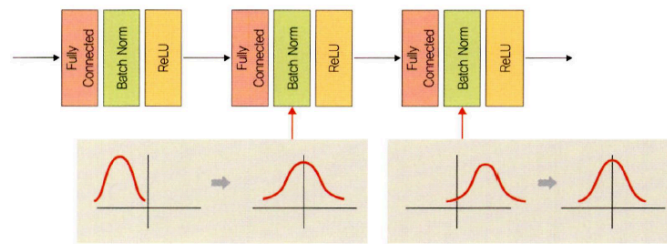
입력 [100, 1, 28, 28]
layer1: Conv2d(1→32, k=3, p=1) → BatchNorm2d(32) → ReLU → MaxPool2d(2, 2) → [100, 32, 14, 14]
layer2: Conv2d(32→64, k=3) → BatchNorm2d(64) → ReLU → MaxPool2d(2) → [100, 64, 6, 6]
view(out.size(0), -1) → [100, 2304]
fc1 (2304 → 600) → Dropout2d(0.25) → fc2 (600 → 120) → fc3 (120 → 10) → [100, 10]

```

8-2. 번호별 설명

번호	항목	설명
①	nn.Sequential	init() 에서 네트워크 모델을 정의할 뿐 아니라, forward() 에서 구현될 순전파를 계층(layer) 형태로 가독성 있게 작성 가능. 계층을 차례로 쌓아 $Wx + b$ 같은 수식과 활성화 함수를 연결. 데이터가 각 계층을 순차적으로 지나갈 때 좋음. 정리하면 여러 계층을 하나의 컨테이너에 구현하는 방법
②	nn.Conv2d(...)	합성곱층: 커널(필터)이라는 $n \times m$ 행렬이 높이×너비 이미지를 처음부터 끝까지 훑으면서 원소끼리 곱한 후 더한 값 출력. 커널은 일반적으로 3×3 , 5×5
③	nn.BatchNorm2d(32)	학습 과정에서 각 배치 단위별로 데이터가 다양한 분포를 가지더라도 평균과 분산을 이용해 정규화 . 배치·계층마다 입력 분포가 달라도 정규화로 가우시안 형태 → 평균 0, 표준편차 1 로 조정 (그림 5-28: Fully Connected → Batch Norm → ReLU 반복)
④	nn.MaxPool2d(kernel_size=2, stride=2)	이미지 크기 축소. 합성곱층 출력을 입력으로 받아 activation map 크기를 줄이거나 특정 데이터를 강조. 종류: 최대 풀링(max), 평균 풀링(average), 최소 풀링(min)
⑤	nn.Linear(in_features=6466, out_features=600)	클래스 분류를 위해 이미지 형태 데이터를 배열 형태로 변환. Conv2d 하이퍼파라미터(패딩·스트라이드)에 따라 출력 크기가 달라지며, 줄여든 출력이 분류를 담당하는 완전연결층 으로 전달
⑥	out.view(out.size(0), -1)	합성곱층 → 완전연결층으로 넘어가므로 데이터 형태를 1차원으로. $out.size(0) = 100$ (배치) → $(100, ?)$ 크기로 변경. -1은 행 수는 정확히 알지만 열 수를 모를 때 사용

♥ 그림 5-28 BatchNorm2d



8-3. nn.Conv2d 파라미터

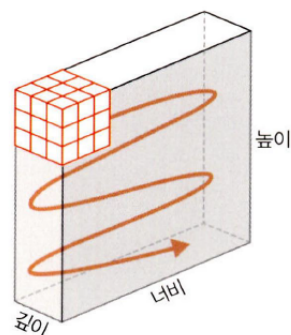
```
nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
```

파라미터	의미
㉠ in_channels	입력 채널의 수. 흑백 이미지 1, RGB 이미지 3
㉡ out_channels	출력 채널의 수 (= 필터 개수)
㉢ kernel_size	커널 크기 (논문에 따라 필터). 이미지 특징을 찾아내기 위한 공용 파라미터이며, CNN에서 학습 대상은 필터 파라미터 . 커널은 입력 데이터를 스트라이드 간격으로 순회하며 합성곱 계산. kernel_size=3 → (3, 3) 정사각형, 직사각형은 (3, 5)처럼 지정
㉣ padding	패딩 크기. 출력 크기를 조정하기 위해 입력 데이터 주위에 0을 채움 . 패딩 값이 클수록 출력 크기도 커짐

채널이란?

- 흑백 이미지: $w \times h$ 행렬 (w 너비, h 높이)
- 컬러 이미지: Red, Green, Blue 세 개의 채널 → $w \times h \times c$ 행렬 (c 채널 수)
- 3차원으로 생각하면 **채널 = 깊이(depth)**

♥ 그림 5-27 채널



8-4. nn.MaxPool2d 파라미터

```
nn.MaxPool2d(kernel_size=2, stride=2)
```

파라미터	의미
㉠ kernel_size	m×n 행렬로 구성된 가중치 (풀링 영역 크기)
㉡ stride	입력 데이터에 커널(필터)을 적용할 때 이동할 간격 . 스트라이드가 커지면 출력 크기는 작아짐

8-5. nn.Linear 파라미터

```
nn.Linear(in_features=64*6*6, out_features=600)
```

파라미터	의미
㉠ in_features	입력 데이터의 크기. Conv2d·MaxPool2d는 이미지 데이터를 입력받아 처리했지만, 그 출력을 완전 연결층으로 보내려면 1차원으로 변경 해야 함. 공식이 Conv2d와 MaxPool2d가 다르므로 매 계층을 따라가면서 계산
㉡ out_features	출력 데이터의 크기

8-6. 출력 크기 계산 공식

Conv2d 계층

항목	내용
공식	출력 크기 = $(W - F + 2P) / S + 1$
W	입력 데이터의 크기 (input_volume_size)
F	커널 크기 (kernel_size)
P	패딩 크기 (padding_size)
S	스트라이드 (strides). 명시되어 있지 않으면 기본값 (1, 1)

MaxPool2d 계층

항목	내용
공식	출력 크기 = IF / F
IF	입력 필터의 크기 (input_filter_size, 바로 앞 Conv2d의 출력 크기)
F	커널 크기 (kernel_size)

교재의 계산 (p.197)

계층	교재 계산	교재가 적은 출력 형태
첫 번째 Conv2d(k=3, p=1)	$(784 - 3 + 2 \times 1) / 1 + 1 = 784$	[32, 784, 784] (32 = out_channels)
첫 번째 MaxPool2d(k=2)	$784 / 2 = 392$	[32, 392, 392]

💡 padding=1인 3×3 합성곱은 크기를 **유지**(28→28), padding 없는 3×3 합성곱은 **2씩 깎임**(14→12), MaxPool(2)는 **반으로**(28→14, 12→6). 이 세 규칙만 기억하면 어떤 CNN이든 in_features를 암산할 수 있다

8-7. DNN vs CNN 비교

항목	FashionDNN	FashionCNN
입력 처리	처음부터 784로 펼침 (공간 구조 손실)	28×28 2차원 그대로 합성곱
특성 추출	없음 (완전연결층만)	Conv2d + BatchNorm + ReLU + MaxPool × 2
펼치는 시점	맨 처음 (view(-1, 784))	합성곱·풀링 끝난 뒤 (view(out.size(0), -1))
완전연결층	784 → 256 → 128 → 10	2304 → 600 → 120 → 10
Dropout	nn.Dropout(0.25)	nn.Dropout2d(0.25)
활성화 함수 위치	forward()에서 F.relu()	init() 에서 nn.ReLU() (Sequential 안)

9. 코드 5-10. 합성곱 네트워크를 위한 파라미터 정의

- 코드 5-7과 **같은 구조**, 모델만 FashionDNN → **FashionCNN**

항목	코드
학습률	learning_rate = 0.001
모델	model = FashionCNN(); model.to(device)
손실 함수	criterion = nn.CrossEntropyLoss()
옵티마이저	optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

print(model) 출력 — 합성곱 네트워크 구조

```
FashionCNN(
  (layer1): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (layer2): Sequential(
    (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (fc1): Linear(in_features=2304, out_features=600, bias=True)
  (drop): Dropout2d(p=0.25, inplace=False)
  (fc2): Linear(in_features=600, out_features=120, bias=True)
  (fc3): Linear(in_features=120, out_features=10, bias=True)
)
```

출력에서 읽을 수 있는 것:

계층	기본값	의미
Conv2d	stride=(1, 1)	스트라이드 기본값 1
Conv2d (layer2)	padding 표시 없음	기본값 0 → 크기가 2 줄어듦 (14 → 12)
BatchNorm2d	eps=1e-05, momentum=0.1, affine=True	분산에 더하는 작은 값 / 이동 평균 갱신 비율 / 학습 가능한 스케일·시프트
MaxPool2d	padding=0, dilation=1, ceil_mode=False	패딩 없음, 확장 없음, 나머지는 버림

계층	기본값	의미
fc1	in_features= 2304	64×6×6 — 8-6에서 계산한 값과 일치 ✓

10. 코드 5-11. 모델 학습 및 성능 평가

- 코드 5-8과 **완전히 동일한 학습 루프** (model 객체만 CNN으로 바뀜)
- 학습 5단계 → 50번마다 테스트 정확도 → 500번마다 출력, 구조 그대로

실행 결과

Iteration	Loss	Accuracy
500	0.4393	87.87%
1000	0.3259	88.10%
1500	0.3012	88.39%
2000	0.1755	89.63%
2500	0.1378	89.57%
3000	0.2483	88.95%

11. DNN vs CNN 결과 비교 및 결론

Iteration	DNN (코드 5-8)	CNN (코드 5-11)	차이
500	82.71%	87.87%	+5.16
1000	84.40%	88.10%	+3.70
1500	84.41%	88.39%	+3.98
2000	85.06%	89.63%	+4.57
2500	86.26%	89.57%	+3.31
3000	86.48%	88.95%	+2.47

- 심층 신경망과 비교하여 정확도가 **약간 높다**
- 별 차이가 없기 때문에 간편한 심층 신경망만 사용해도 무난할 것 같지만, 실제로 **이미지 데이터가 많아지면** 단순 심층 신경망으로는 **정확한 특성 추출 및 분류가 불가능** → 합성곱 신경망을 생성할 수 있도록 학습해야 함